Proyecto Final

Procesamiento en Tiempo Real con Docker, Kafka y Spark

Alumno: David Rafael

Módulo: Módulo 5. Procesamiento en Tiempo Real con Apache Kafka y Spark

Sesión: Sesión 3 — Taller Integrador de un Pipeline en Tiempo Real

Docente: Layla Scheli

Parte 1: Conceptos básicos

Docker

¿Qué es un contenedor y cómo se diferencia de una máquina virtual?

Un contenedor empaqueta una aplicación junto con sus dependencias, pero comparte el kernel del sistema operativo del host. Esto lo hace ligero y rápido de iniciar (segundos). Una máquina virtual, en cambio, emula un sistema operativo completo sobre un hipervisor, incluyendo su propio kernel, lo que la hace más pesada, más lenta de arrancar y con mayor consumo de recursos. En este proyecto, cada servicio (Kafka, Zookeeper, Jupyter) corre en su propio contenedor ligero dentro de la misma máquina.

¿Qué son los volúmenes en Docker y para qué sirven?

Los volúmenes son espacios de almacenamiento que existen fuera del ciclo de vida del contenedor. Permiten que los datos persistan aunque el contenedor se borre o se recree. En este proyecto se usó un bind mount (`./spark-app:/home/jovyan/work`) para que los notebooks creados dentro de Jupyter se guardaran directamente en el disco del host, y no se perdieran al limpiar los contenedores.

Explica la diferencia entre docker run y docker compose.

`docker run` levanta un único contenedor a partir de una imagen, especificando sus opciones directamente en la línea de comandos. `docker compose` permite definir y levantar varios servicios relacionados (por ejemplo, Zookeeper, Kafka y Jupyter) a partir de un único archivo `docker-compose.yml`, incluyendo su red y sus dependencias entre sí. Es reproducible y mucho más práctico cuando el proyecto tiene múltiples servicios que deben convivir en la misma red.

Kafka

¿Qué es un topic en Kafka?

Un topic es una categoría o canal con nombre dentro de Kafka donde los productores (producers) escriben mensajes y de donde los consumidores (consumers) los leen. Funciona como un buzón: quien escribe y quien lee nunca se comunican directamente entre sí, solo con Kafka. En este proyecto se usó el topic `actividad-topic`.

¿Cuál es la función de Zookeeper dentro de un clúster Kafka?

Zookeeper coordina el clúster de Kafka: mantiene metadatos sobre qué brokers existen, ayuda a elegir el líder de cada partición y gestiona la configuración distribuida. Las versiones más recientes de Kafka están migrando hacia KRaft, que elimina la dependencia de Zookeeper, pero en este proyecto se usó la arquitectura clásica con Zookeeper como coordinador.

Explica qué significan “particiones” y “replication factor” en un topic.

Las particiones dividen un topic en varias partes, lo que permite paralelismo: distintos consumidores pueden leer distintas particiones al mismo tiempo. El replication factor indica cuántas copias de cada partición existen en brokers distintos, para tolerar fallas (si un broker cae, otro tiene la copia). En este proyecto se creó `actividad-topic` con 1 partición y replication factor de 1, suficiente para un entorno de una sola instancia de Kafka.

Spark

¿Qué es Spark y cuál es su propósito?

Apache Spark es un motor de procesamiento distribuido diseñado para trabajar con grandes volúmenes de datos, tanto en modo batch como en streaming. Su propósito es tomar datos de distintas fuentes (archivos, bases de datos, colas de mensajes como Kafka), transformarlos y llevarlos a un destino final, de forma escalable y más rápida que herramientas anteriores como MapReduce.

Diferencia entre RDD, DataFrame y Dataset.

RDD (Resilient Distributed Dataset) es la abstracción de más bajo nivel en Spark: una colección distribuida de objetos, sin esquema ni optimizaciones automáticas. DataFrame es una capa superior, organizada en columnas con nombre y tipo, similar a una tabla, que Spark puede optimizar internamente mediante su motor Catalyst. Dataset combina lo mejor de ambos (tipado fuerte + optimización), pero solo existe en Scala y Java; en Python (PySpark) se trabaja siempre con DataFrames, como se hizo en este proyecto.

¿Qué es Spark Streaming y para qué se utiliza?

Spark Streaming (en su versión moderna, Structured Streaming) permite procesar datos que llegan de forma continua, como los mensajes de un topic de Kafka, en lugar de datos estáticos ya almacenados. Internamente divide el flujo en micro-lotes (`batches`) que se procesan casi en tiempo real. En este proyecto se usó para leer los mensajes de `actividad-topic` y mostrarlos conforme llegaban, mediante `readStream` y `writeStream`.

Parte 2: Actividad práctica con Docker + Kafka + Spark

Ejercicio 1: Levantar los servicios

Se definió un archivo `docker-compose.yml` con tres servicios: Zookeeper, Kafka y Jupyter (con PySpark preinstalado). Los tres comparten una red interna llamada `app-network`.

```
services:
zookeeper:
image: confluentinc/cp-zookeeper:7.5.0
container_name: zookeeper
ports:
- "2181:2181"
environment:
ZOOKEEPER_CLIENT_PORT: 2181
ZOOKEEPER_TICK_TIME: 2000
networks:
- app-network

kafka:
image: confluentinc/cp-kafka:7.5.0
container_name: kafka
depends_on:
- zookeeper
ports:
- "9092:9092"
- "29092:29092"
environment:
KAFKA_BROKER_ID: 1
KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092,PLAINTEXT_HOST://0.0.0.0:29092
KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092,PLAINTEXT_HOST://localhost:29092
KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
networks:
- app-network

jupyter:
image: jupyter/pyspark-notebook:python-3.10
container_name: jupyter
ports:
- "8888:8888"
- "4040:4040"
volumes:
- ./spark-app:/home/jovyan/work
environment:
JUPYTER_ENABLE_LAB: "yes"
command: start-notebook.sh --NotebookApp.token=''
depends_on:
- kafka
networks:
- app-network

networks:
app-network:
driver: bridge
```

Se levantaron los servicios y se verificó su estado con:

```
docker compose up -d
docker compose ps
```

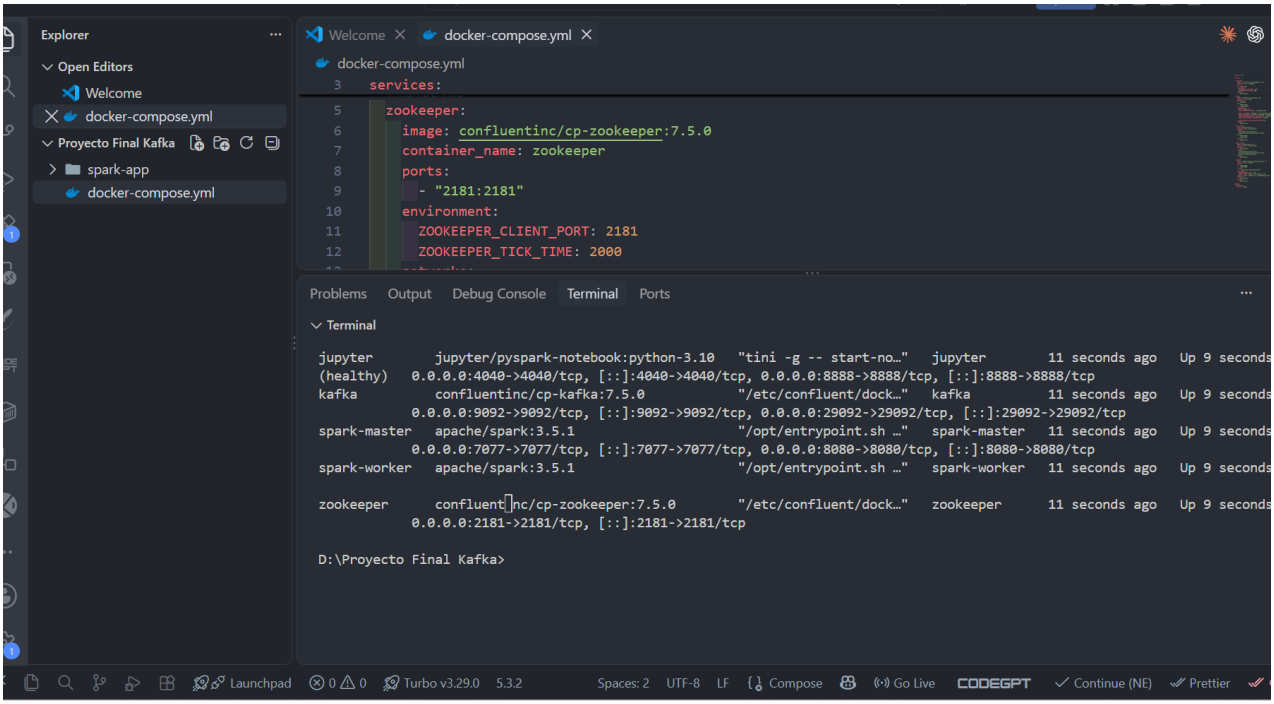


Figura 1. Los tres servicios (jupyter, kafka, zookeeper) corriendo con estado "Up".

Ejercicio 2: Crear un nuevo topic

Dentro del contenedor de Kafka se creó el topic `actividad-topic` con 1 partición y replication factor de 1, y se verificó con `--list` y `--describe`.

```
docker exec -it kafka bash

kafka-topics --bootstrap-server kafka:9092 --create --topic actividad-topic --partitions 1
--replication-factor 1
kafka-topics --bootstrap-server kafka:9092 --list
kafka-topics --bootstrap-server kafka:9092 --describe --topic actividad-topic
```

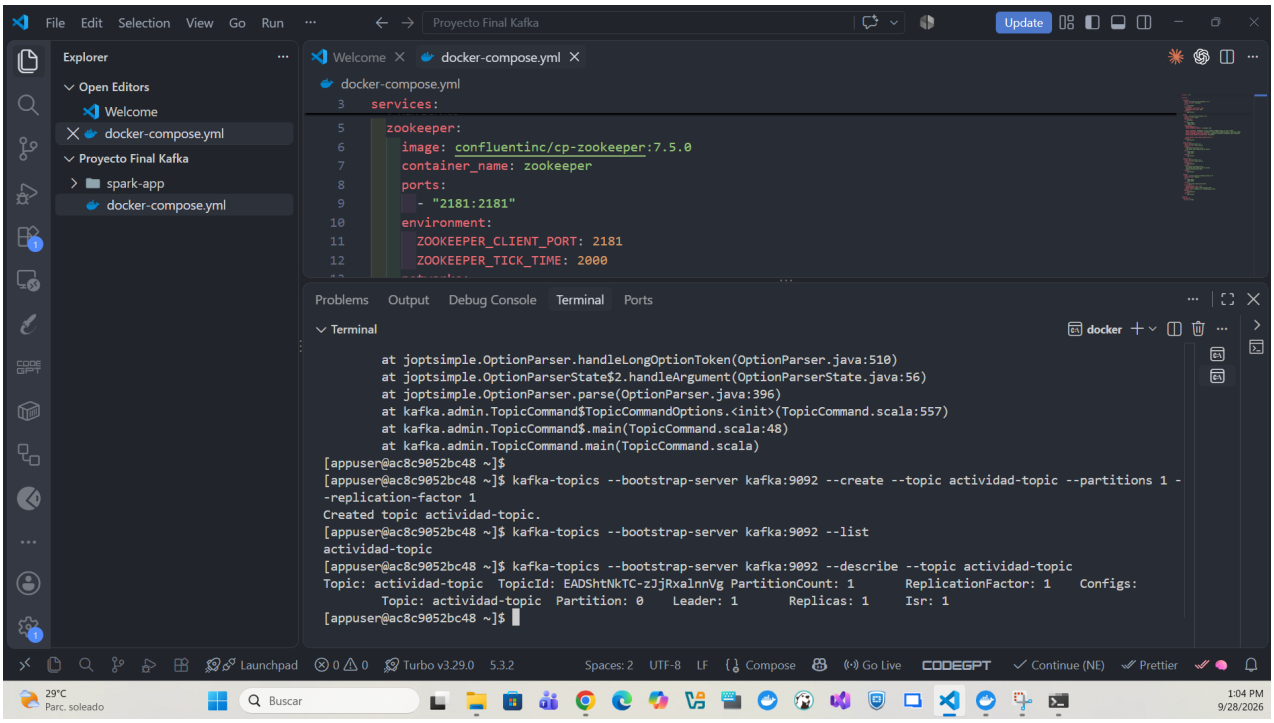


Figura 2. Topic creado y verificado: PartitionCount: 1, ReplicationFactor: 1.

Ejercicio 3: Enviar y leer mensajes

Se abrieron dos terminales dentro del contenedor de Kafka: una con el consumer de consola, en modo escucha, y otra con el producer, desde donde se enviaron 5 mensajes de prueba.

```
# Terminal 1 (consumer)
kafka-console-consumer --bootstrap-server kafka:9092 --topic actividad-topic --from-beginning

# Terminal 2 (producer)
kafka-console-producer --bootstrap-server kafka:9092 --topic actividad-topic
> mensaje 1
> mensaje 2
> mensaje 3
> mensaje 4
> mensaje 5
```

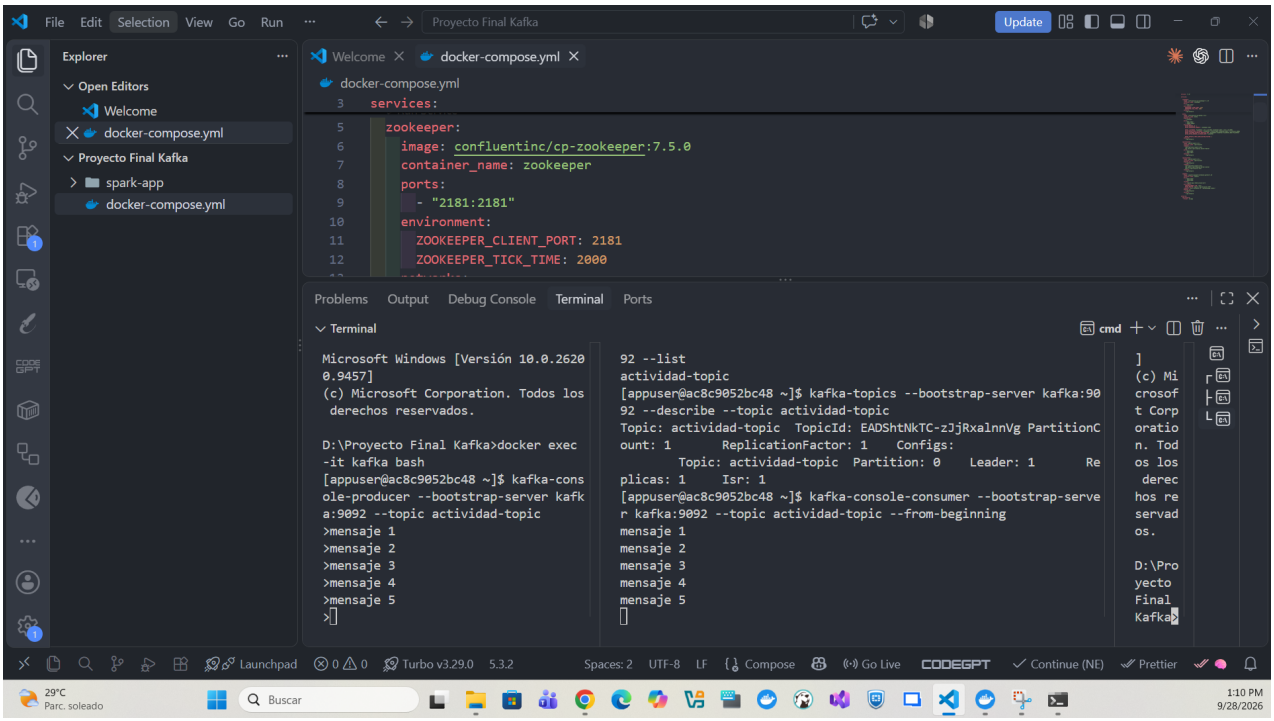


Figura 3. Producer (izquierda) enviando los 5 mensajes; consumer (derecha) recibiéndolos en el mismo orden.

Ejercicio 4: Spark Streaming básico

Se creó un notebook en Jupyter que usa Spark Structured Streaming para suscribirse a `actividad-topic` y mostrar los mensajes conforme llegan. Se usó `foreachBatch` en lugar del sink `console` estándar, porque este último imprime en la salida del proceso Java de Spark y no en la celda del notebook; con `foreachBatch` el procesamiento vuelve a Python y sí se muestra directamente en Jupyter.

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
.appName("ActividadStreaming") \
.config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-10_2.12:3.4.0") \
.getOrCreate()

df = spark.readStream.format("kafka") \
.option("kafka.bootstrap.servers", "kafka:9092") \
.option("subscribe", "actividad-topic") \
.option("startingOffsets", "earliest") \
.load()

mensajes = df.selectExpr("CAST(value AS STRING) AS mensaje")

def procesar(batch_df, epoch_id):
print(f"--- Batch {epoch_id} ---")
batch_df.show(truncate=False)

mensajes.writeStream \
.foreachBatch(procesar) \
.outputMode("append") \
.start() \
.awaitTermination()
```

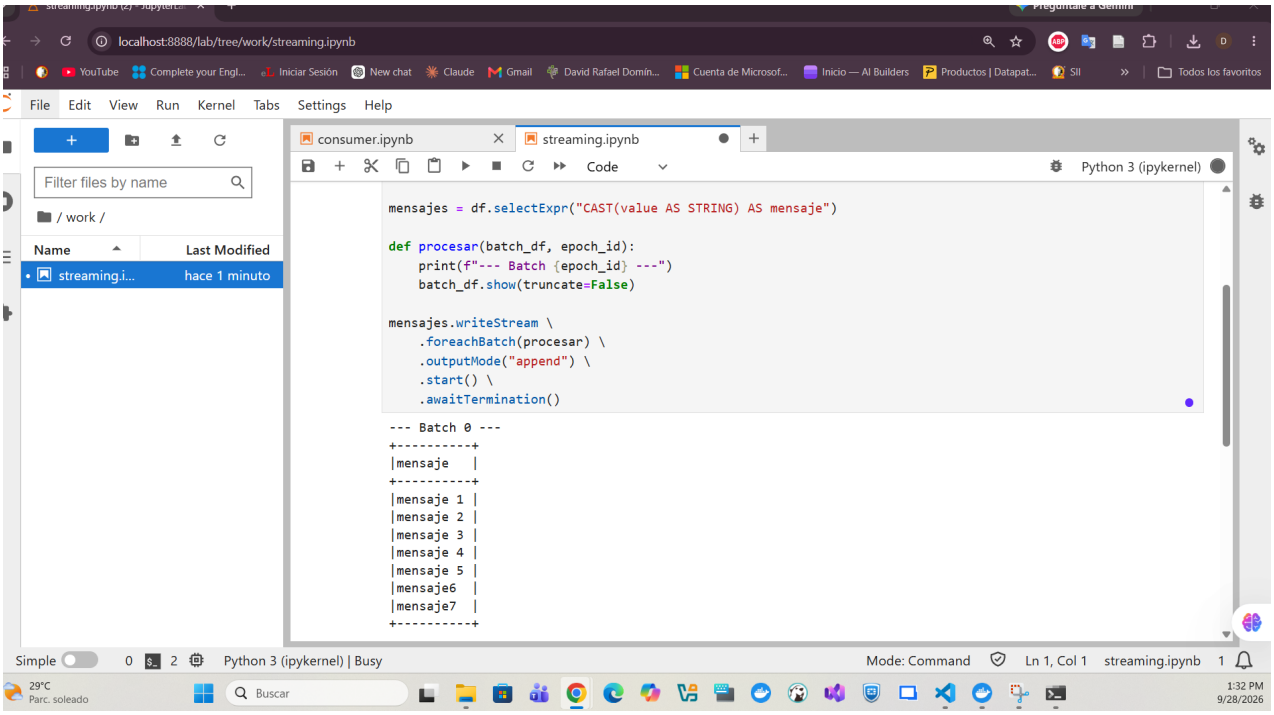


Figura 4. Al iniciar el stream, Batch 0 trae los 7 mensajes ya existentes en el topic (startingOffsets=earliest).

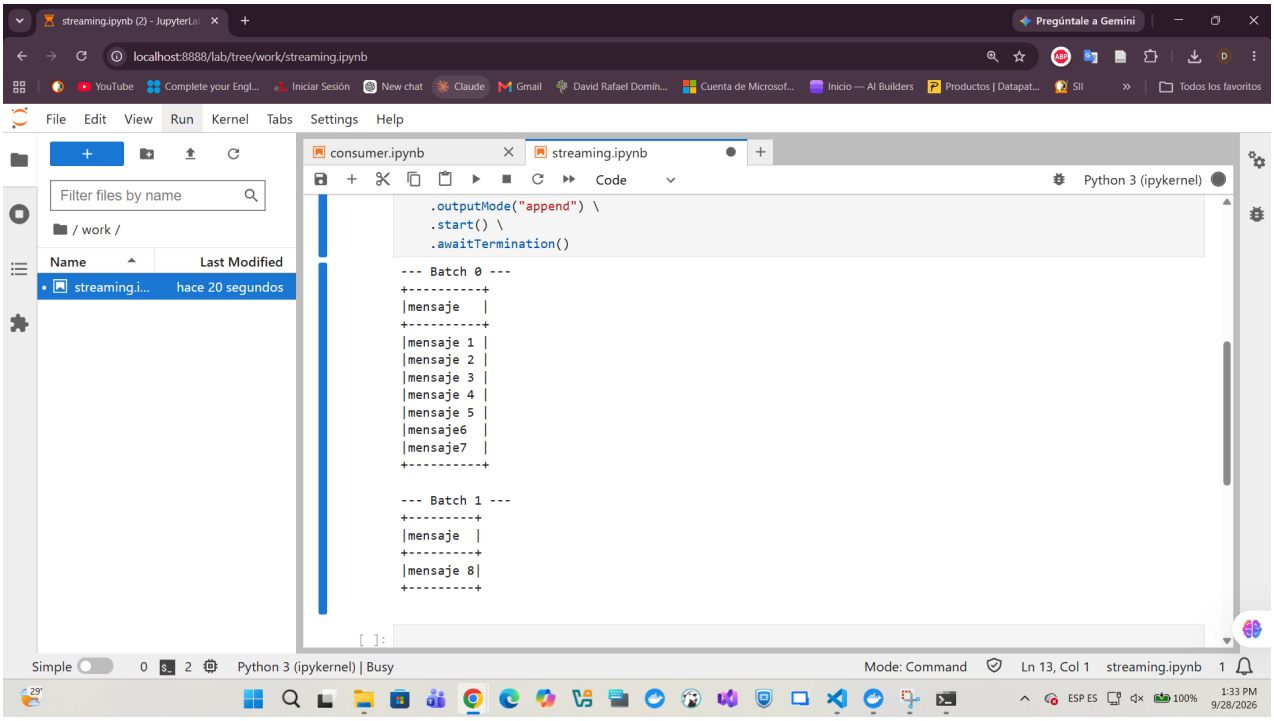


Figura 5. Con el stream corriendo, al enviar un mensaje nuevo desde el producer aparece automáticamente Batch 1 con “mensaje 8” — procesamiento en tiempo real.

Ejercicio 5: Limpieza de recursos

Finalmente se detuvo el stream de Spark y se eliminaron los contenedores, la red y el volumen creados para esta práctica, y luego se limpiaron las imágenes y volúmenes ya no utilizados.

```
docker compose down -v
```

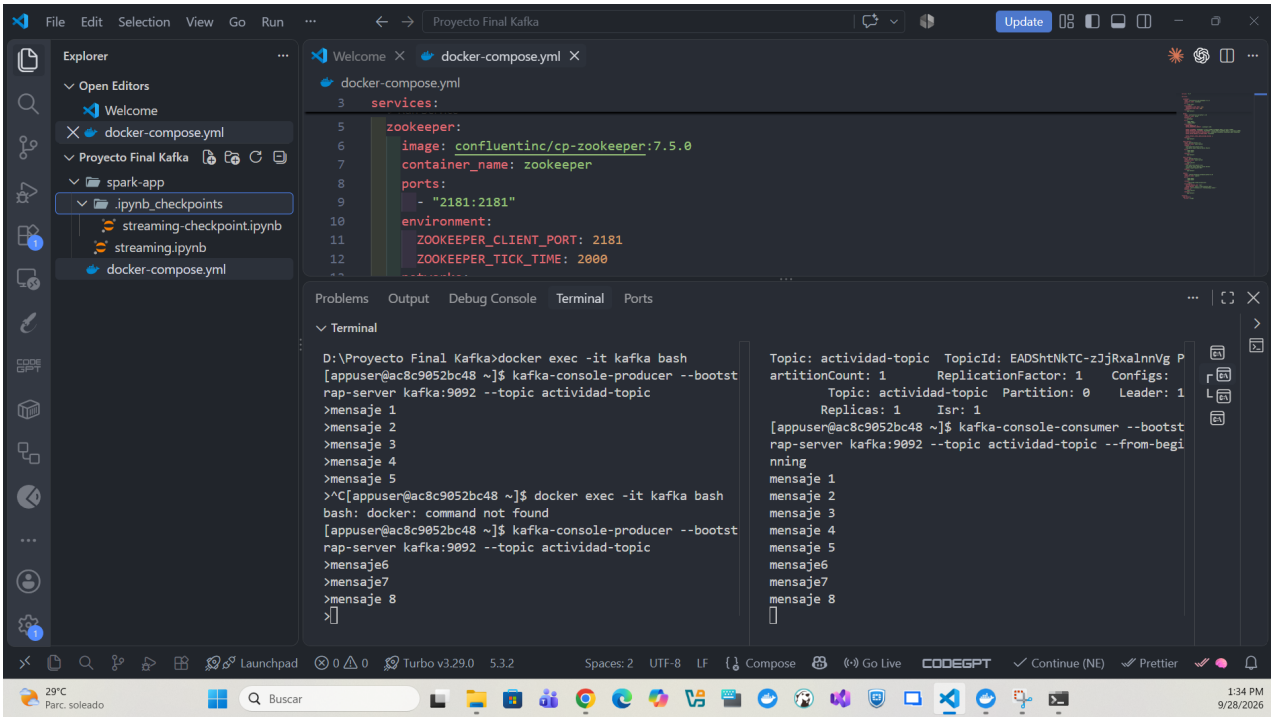


Figura 6. Los 5 contenedores (zookeeper, kafka, jupyter, spark-master, spark-worker) y la red del proyecto fueron eliminados.

```
docker image prune -a
docker volume prune

docker ps -a
docker images
docker volume ls
```

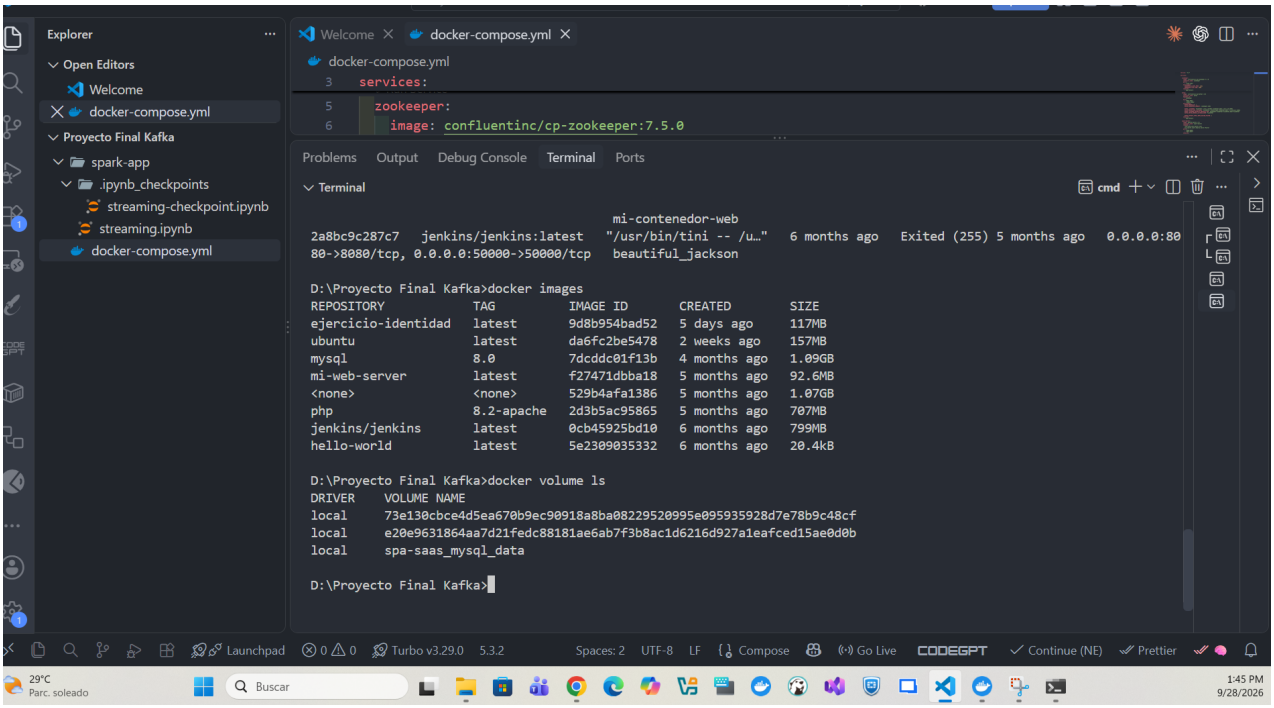


Figura 7. Verificación final: ya no quedan imágenes ni volúmenes relacionados con Kafka, Zookeeper, Spark o Jupyter.

Parte 3: Comunicación de aprendizajes

El artículo con la experiencia completa de esta actividad —lo aprendido sobre Docker, Kafka y Spark, los desafíos enfrentados durante el desarrollo y la reflexión sobre el procesamiento de datos en tiempo real— fue publicado en Medium:

Enlace al artículo: https://medium.com/@dodamivid/aprendiendo-a-domar-kafka-y-spark-a-punta-de-errores-4c69945ff1e8

El repositorio de GitHub con este documento, el archivo docker-compose.yml y el notebook de Spark Streaming se encuentra en:

Repositorio: https://github.com/dodamivid/proyecto-kafka-spark